S500 Agentic Drone Build Guide

Nov 6, 2026 · @Antonio

Overview

The Holybro S500 V2 runs ArduCopter 4.7.2 on its Pixhawk 6C. An RPLidar C1 plugs straight into the Pixhawk, so obstacle avoidance always runs on the flight controller. Targets come from the ground station over the SiK radio, and from the Raspberry Pi 5 over a UART. **QGroundControl** (stable release) on Ubuntu is the ground station for setup, monitoring and flying.

Who does what

- **Pixhawk 6C + ArduCopter:** always the pilot. Runs GUIDED mode, BendyRuler path planning from the lidar, and every failsafe.
- **Raspberry Pi 5:** mission manager and camera. Sends GPS targets when far away, body-relative moves when close.
- **Ground station (Ubuntu laptop):** QGroundControl shows the drone and handles setup; MAVProxy shares the radio link between QGC and your agent scripts.
- **RC transmitter (SBUS receiver):** final manual override at all times.

Tools

Job	Tool
Simulation, building, script development	ArduPilot SITL + MAVProxy
Drone setup, calibration, parameters, motor test	QGroundControl
Starting tune values from prop size and battery	ArduPilot Methodic Configurator (Phase 5.7)
Sharing the radio between QGC and scripts	MAVProxy router (Phase 8)
Optional, on Windows: lidar radar view, tuning helper, log analysis	Mission Planner

Your scripts always connect to `udpin:0.0.0.0:14551`. In simulation and in the field, MAVProxy feeds that port, so the same scripts run unchanged in both.

Use the **stable** QGroundControl release, not Daily: the Daily build's firmware page had no ArduPilot option.

Order of work. Simulation first, so every command and parameter is proven before it touches the aircraft. Each phase ends with a check that must pass before the next one.

Safety rules for every bench phase

- Propellers **off** for everything until Phase 9. That includes "just a quick motor test".
- Flight battery disconnected while wiring. Power the Pixhawk from USB for configuration.
- Never power the Pi 5 from a Pixhawk port.
- One change at a time, then test. After each phase that passes, save the parameters (QGC: Parameters → Tools → Save to file) into your repo's `params/` folder and commit.

Progress

Phase	Status	Notes
0 Parts and pre-checks	Done	PX4 parameters backed up, motor mapping recorded
1 Simulation	In progress	Build and lidar working; re-test avoidance with `GUID_OPTIONS` = 64
2 Firmware choice	Done	`Copter/stable/Pixhawk6C/arducopter.apj`
3 Flashing	Done	Flashed with the uploader; reset and save parameters if not done yet
4 Wiring	To do	New port plan: SiK TELEM1, lidar TELEM2, Pi TELEM3
5 Base setup in QGroundControl	To do	
6 Lidar and avoidance	To do	
7 Raspberry Pi 5	To do	
8 Ground station	To do	
9 Flight tests	To do	

Phase 0: Parts and pre-checks

Part	Notes
Holybro S500 V2 frame, motors, ESCs	Four ESCs on DShot600, 10-inch props, 4S battery
Holybro Pixhawk 6C (not Mini)	The Mini has no TELEM3, which this plan uses for the Pi
GPS/compass module	On GPS1
Holybro SiK radio pair (915 MHz)	Air unit on TELEM1, ground unit on the laptop's USB
Slamtec RPLidar C1	On TELEM2; keep its USB adapter board for bench tests
SBUS RC receiver + transmitter	On RC IN; manual override
Raspberry Pi 5 + microSD (32 GB+)	On TELEM3
5V 5A BEC (4S input)	Dedicated supply for the Pi 5
JST-GH 6-pin cables, Dupont jumpers	For TELEM2 → lidar and TELEM3 → Pi

Pre-checks

- [x] ~~Read the Pixhawk 6C serial number. If its middle block is **001**, TELEM3 pins 4 and 5 are I2C; fine here, because the Pi uses only TX, RX and GND, but never connect anything to those two pins.~~
- [x] ~~Back up the PX4 parameters before flashing.~~
- [x] ~~Record the PX4 motor mapping: ESCs on AUX 1–4 with DShot600, AUX1 = Motor 3, AUX2 = Motor 2, AUX3 = Motor 4, AUX4 = Motor 1.~~
- [x] ~~Note the prop size and cell count: 10-inch, 4S for the stock kit.~~
- [x] ~~Test the lidar alone on its USB adapter board with Slamtec's RoboStudio, so you know the sensor works before wiring it in.~~

Phase 1: Simulation on Ubuntu

ArduPilot's simulator (SITL) fakes an RPLidar surrounded by virtual posts, so avoidance and your scripts can be tested with no hardware. Everything here runs on your laptop in Ubuntu, not on the Pi.

1.1 One-time shell setup. Two shortcuts used throughout this guide: `$REPO` for your repo and `apenv` to activate the ArduPilot Python environment.

```
echo 'export REPO=$HOME/Documents/2026/usyd/yr2sem2/joey-
tron/tronV2adrupilot' >> ~/.bashrc
echo "alias apenv='source ~/venv-ardupilot/bin/activate'" >> ~/.bashrc
. ~/.bashrc
```

Don't make the ArduPilot venv activate automatically in every terminal. It carries setuptools 84, and colcon (your ROS 2 Jazzy builds) needs a version below 80. Activate it with `apenv` only where you're doing ArduPilot work.

1.2 Repo layout (done). Your repo `TronV2ardrupilot` holds everything for this build, with ArduPilot as a submodule pinned to Copter-4.7.2:

```
tronV2adrupilot/
├── ardupilot/   submodule, Copter-4.7.2
├── gcs/         ground station scripts: goto.py (SITL), target.py (field)
├── pi/          Pi scripts: nudge.py, mission manager
├── params/      .params files, one per passing phase
├── firmware/    the .apj you flashed (ignored by Git)
└── logs/        flight logs (ignored by Git)
```

Check the submodule is on the right version. Your prompt shows `APMrover2-stable` because several tags can point at one commit; what matters is that Copter-4.7.2 is among them:

```
git -C $REPO/ardupilot tag --points-at HEAD | grep Copter-4.7.2
```

If that prints nothing, pin it and rebuild:

```
cd $REPO/ardupilot
git checkout Copter-4.7.2
git submodule update --init --recursive
cd $REPO && git add ardupilot && git commit -m "Pin ArduPilot to Copter-
4.7.2"
```

1.3 Prerequisites and build (done). For reference, or if you set up another machine:

```
cd $REPO/ardupilot
Tools/environment_install/install-prereqs-ubuntu.sh
apenv
./waf configure --board sitl
./waf copter
```

Run the install script without `-y` and answer **N** to "Make ArduPilot venv default" and to "Install ArduPilot STM32 toolchain" (you flash the official `.apj`, so you don't need the compiler). If it stalls on wxPython from extras.wxpython.org, press Ctrl+C: wxPython 4.2.1 is already installed system-wide, and the venv can see it.

1.4 Start SITL with the simulated lidar and posts. In a new terminal:

```
apenv
cd $REPO/ardupilot/ArduCopter
../Tools/autotest/sim_vehicle.py -v ArduCopter -A --serial5=sim:rplidara2 -l
51.8752066,14.6487840,54.15,0 --console --map --out=udp:127.0.0.1:14551
```

The first time ever, add `-w` to wipe the simulated parameters to defaults. Never use `-w` after that, or your settings are lost.

The simulator puts its lidar on SERIAL5. On the real drone the lidar is on SERIAL2 (Phase 4), so never load a SITL parameter file onto the drone.

1.5 MAVProxy commands. Type these at the `STABILIZE>` prompt in that same terminal, **one line at a time**, waiting for each reply. Pasting several lines at once glues them together.

```
script /tmp/post-locations.scr
param set SERIAL5_PROTOCOL 11
param set PRX1_TYPE 5
param set AVOID_ENABLE 7
param set AVOID_MARGIN 2
param set OA_TYPE 1
param fetch
```

Wait for `Received ... parameters`, then:

```
param set OA_BR_LOOKAHEAD 5
param set OA_MARGIN_MAX 2
param set OA_DB_OUTPUT 3
param set GUID_OPTIONS 64
reboot
```

The first line draws the posts on the map as blue circles. `GUID_OPTIONS` = 64 is what makes GUIDED mode use BendyRuler; without it GUIDED flies straight through obstacles (Guided mode options). Changing `OA_TYPE` or `GUID_OPTIONS` needs the reboot. `OA_DB_OUTPUT` = 3 shows detected obstacles on the map as red `[OA_DB]` labels.

1.6 Manual test. After the reboot reconnects and the console shows GPS and EKF ready:

```
mode guided
arm throttle
takeoff 10
```

Right-click the map at a point straight past a post, about 20–30 m away, and choose **Fly To**. It should bend around the post, keeping about 2 m away. To make the dodge obvious, `param set OA_MARGIN_MAX 4`, then set it back to 2.

1.7 The same test from Python. Save as `$REPO/gcs/goto.py`. It arms, takes off and flies to a target, so it is for SITL only:

```
import sys, time, math
from pymavlink import mavutil

conn = sys.argv[1]
lat, lon, alt = (float(v) for v in sys.argv[2:5])

m = mavutil.mavlink_connection(conn)
while True:
    hb = m.recv_match(type="HEARTBEAT", blocking=True)
    if hb.autopilot == mavutil.mavlink.MAV_AUTOPILOT_ARDUPILOTMEGA:
        m.target_system = hb.get_srcSystem()
        m.target_component = hb.get_srcComponent()
        break

m.set_mode("GUIDED")
while not m.motors_armed():
    m.arducopter_arm()
    time.sleep(2)
    m.recv_match(type="HEARTBEAT", blocking=True, timeout=3)

m.mav.command_long_send(m.target_system, m.target_component,
    mavutil.mavlink.MAV_CMD_NAV_TAKEOFF, 0, 0, 0, 0, 0, 0, 0, alt)

while True:
    p = m.recv_match(type="GLOBAL_POSITION_INT", blocking=True)
```

```
    if p.relative_alt / 1000 > alt * 0.9:
        break

m.mav.set_position_target_global_int_send(
    0, m.target_system, m.target_component,
    mavutil.mavlink.MAV_FRAME_GLOBAL_RELATIVE_ALT_INT,
    0b110111111000, int(lat * 1e7), int(lon * 1e7), alt,
    0, 0, 0, 0, 0, 0, 0, 0)

while True:
    p = m.recv_match(type="GLOBAL_POSITION_INT", blocking=True)
    dn = (lat - p.lat / 1e7) * 111320
    de = (lon - p.lon / 1e7) * 111320 * math.cos(math.radians(lat))
    d = math.hypot(dn, de)
    print(f"{d:6.1f} m to go")
    if d < 2:
        break
print("arrived")
```

With SITL from 1.4 freshly started and disarmed, in a second terminal:

```
apenv
cd $REPO/gcs
python3 goto.py udpin:0.0.0.0:14551 <lat> <lon> 10
```

Pick a target behind a post; the map shows the latitude and longitude under the mouse.

Phase 1 passes when: the drone bends around posts in both 1.6 and 1.7, and `goto.py` prints "arrived". Commit `goto.py`.

Phase 2: Firmware choice (done)

Use **ArduCopter 4.7.2, stable, board folder `Pixhawk6C`**, the same version as your SITL build (Copter-4.7.2 announcement).

Folder in `Copter/stable/`	Use it?	Why
`Pixhawk6C`	**Yes**	Standard multirotor firmware; supports DShot600 on the AUX outputs

Folder in `Copter/stable/`	Use it?	Why
`Pixhawk6C-bdshot`	No	Adds bidirectional DShot (motor RPM feedback), which the stock S500 ESCs don't need
`Pixhawk6C-heli`, `Pixhawk6C-bdshot-heli`	No	Traditional single-rotor helicopters

Inside the folder, check `git-version.txt` says Copter 4.7.2, then download `arducopter.apj` (not `arducopter_with_bl.hex`) into `$REPO/firmware/`. The `.apj` is JSON, so you can read its header without flashing:

```
cd $REPO/firmware
python3 -c "import
json;d=json.load(open('arducopter.apj'));print(d['board_id'],
d.get('git_identity'))"
```

`board_id` must be **56**, the Pixhawk 6C's ID in ArduPilot's board definition. If the file won't parse, the download is incomplete.

Phase 3: Flashing (done)

The board was flashed with ArduPilot's uploader from Ubuntu, and QGroundControl now identifies it as `Pixhawk6C [Holybro]` running ArduCopter. The methods are kept here for future updates.

Before any flash: flight battery disconnected, props off, USB cable straight into the laptop, parameters saved.

Option A: Terminal uploader (recommended, what you used)

Close QGroundControl first; it holds the USB port. Start with the Pixhawk unplugged:

```
apenv
cd $REPO/ardupilot
python3 Tools/scripts/uploader.py --port /dev/ttyACM0
$REPO/firmware/arducopter.apj
```

Plug the board in when it says it's waiting for the bootloader. It reports board ID 56, then erases, programs and verifies.

Option B: QGroundControl stable

1. Unplug the Pixhawk and open Vehicle Setup → **Firmware**.
2. Plug it in. In the dialog choose **ArduPilot**, tick **Advanced settings**, pick **Custom firmware file...**, and select `arducopter.apj`.
3. Wait for erase, program and verify before unplugging.

Confirm and reset

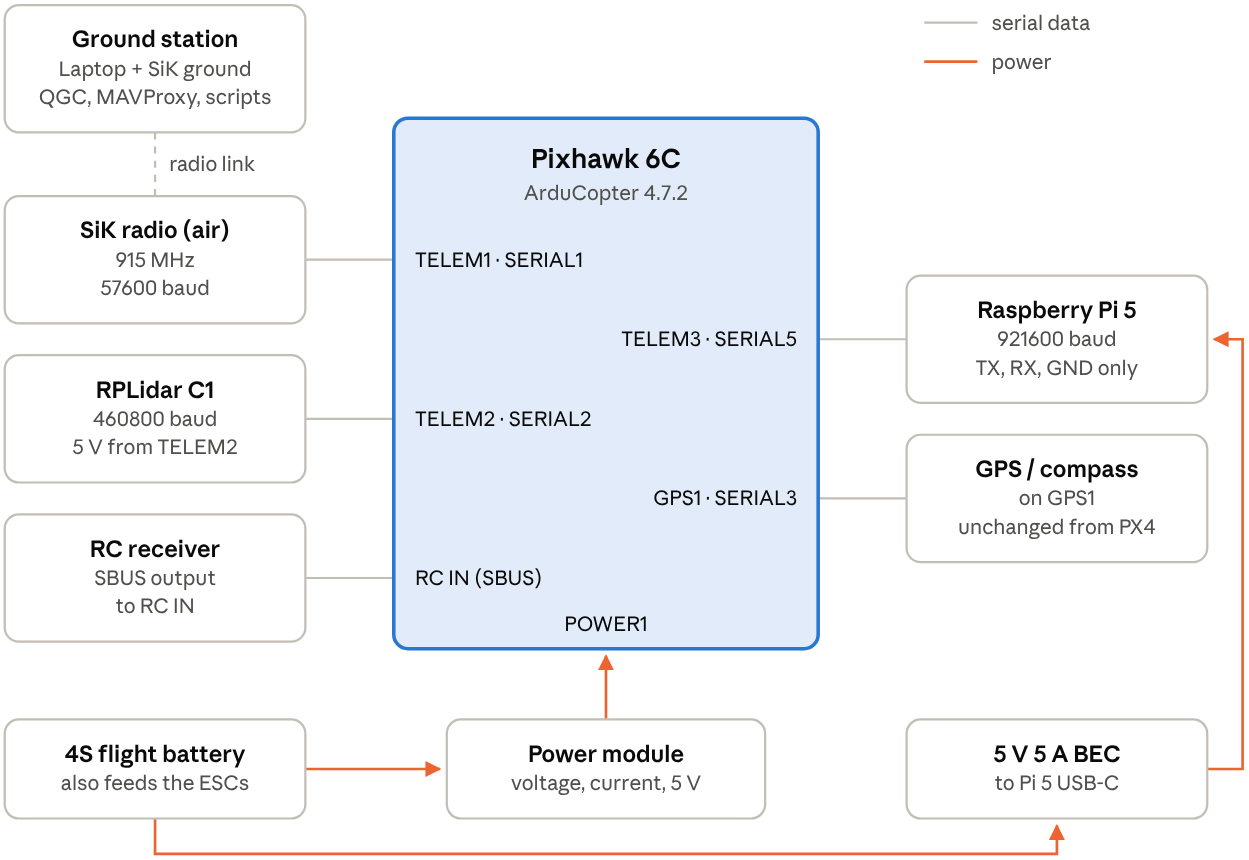
1. Open QGroundControl. It should connect as an ArduPilot vehicle, and the vehicle messages should show `ArduCopter V4.7.2` and `Pixhawk6C`.
2. Reset to defaults: **Parameters → Tools → Reset all to firmware's defaults**, then **Parameters → Tools → Reboot Vehicle**.
3. Save the parameters as `$REPO/params/s500_ardu_00_fresh.params` and commit.

If a flash fails partway, the bootloader survives. Unplug, wait 10 seconds and repeat.

Phase 4: Wiring

Every TELEM port on the 6C is a 6-pin JST-GH with 3.3 V logic, and so are the Pi 5's GPIO and the C1, so no level shifters are needed (Holybro 6C port pinouts). **Wire with the flight battery disconnected and the Pixhawk unpowered.**

Three serial devices on three TELEM ports; the Pi has its own supply



S500 wiring · 6 devices, 3 power paths

Only the lidar draws power from the Pixhawk. The Pi runs from its own 5 A supply and shares just ground with the flight controller.

Port plan

6C port	ArduPilot serial	Device	Protocol	Baud
TELEM1	SERIAL1	SiK radio (air unit)	`SERIAL1_PROTOCOL` = 2 (MAVLink2)	`SERIAL1_BAUD` = 57
TELEM2	SERIAL2	RPLidar C1	`SERIAL2_PROTOCOL` = 11 (Lidar360)	`SERIAL2_BAUD` = 460 (460800)
TELEM3	SERIAL5	Raspberry Pi 5	`SERIAL5_PROTOCOL` = 2 (MAVLink2)	`SERIAL5_BAUD` = 921

6C port	ArduPilot serial	Device	Protocol	Baud
GPS1	SERIAL3	GPS/compass	default	default
RC IN	none	SBUS receiver	detected automatically	

The SERIAL numbers don't follow the TELEM numbers: **TELEM3 is SERIAL5** on this board (ArduPilot Pixhawk 6C page). Getting this wrong is the most common reason a device "doesn't work".

TELEM pinout (all three ports)

Pin	Signal	Use here
1 (red)	+5 V out	Lidar power only, never the Pi
2	TX (out)	→ device RX
3	RX (in)	← device TX
4	CTS	Not connected
5	RTS	Not connected
6	GND	Always connected

SiK radio → TELEM1

Plug the Holybro radio's 6-pin cable straight in. TELEM1 has its own 1.5 A power limit, which covers the radio.

RPLidar C1 → TELEM2

TELEM2 pin	Signal	C1 signal
1	+5 V	VCC
2	TX	RX
3	RX	TX
6	GND	GND

- The C1 has a 5-pin XH2.54 socket with VCC, TX, RX and GND (C1 specifications). Wire by the signal names printed on the board or in Slamtec's datasheet, not by pin position; third-party diagrams disagree on the order.
- The C1 draws about 230 mA. The 6C's ports other than TELEM1 share a combined 1.5 A limit (PX4 6C page), so TELEM2 power is within spec. If you see lidar dropouts, move its VCC and GND to a separate 5 V BEC and keep its ground tied to the Pixhawk's.

Raspberry Pi 5 → TELEM3

TELEM3 pin	Signal	Pi 5 header pin
1	+5 V	**Leave unconnected**
2	TX	Pin 10 (GPIO15, RXD)
3	RX	Pin 8 (GPIO14, TXD)
6	GND	Pin 6 (GND)

Pins 4 and 5 stay unconnected; on early 6C units they are I2C. Power the Pi from a dedicated 5 V 5 A BEC into its USB-C port; ArduPilot advises against powering a companion computer from a TELEM port. The ground wire to Pi pin 6 must still be connected, or the link won't work.

GPS → GPS1, SBUS receiver → RC IN

The GPS stays on GPS1 as it was under PX4. Plug the receiver's SBUS output into **RC IN** with the RC cable that came with the 6C; ArduPilot detects SBUS automatically.

Mounting the lidar

- Flat, with its cable exit pointing to the **rear** (ArduPilot RPLidar page).
- On the S500 the props, arms and GPS mast sit near the scan plane. Mount the C1 on a short mast above the props, with nothing else in its scan plane. Anything it sees all the time is treated as an obstacle (Phase 6 checks this).

Before powering on

- [x] ~~Every TX goes to an RX, every RX to a TX.~~
- [x] ~~No 5 V wire between the Pixhawk and the Pi.~~
- [x] ~~Ground connected Pixhawk ↔ Pi and Pixhawk ↔ lidar.~~
- [x] ~~Nothing on TELEM3 pins 4 and 5.~~
- [x] ~~Every BEC output checked with a multimeter before connecting it.~~
- [x] ~~Props off.~~

Phase 5: Base setup in QGroundControl

All in QGroundControl on Ubuntu, Pixhawk on USB. Props off. Setup pages are under **Vehicle Setup** (Vehicle Configuration in newer versions); page names can differ slightly between QGC versions. Parameters are set in **Parameters**: search the name, click it, type the value, **Save**. Reboot with **Parameters → Tools → Reboot Vehicle** when a step says so.

5.1 Frame. Frame page → Quad, X. That sets:

```
FRAME_CLASS = 1
FRAME_TYPE  = 1
```

5.2 Serial ports (Phase 4 port plan). Reboot afterwards.

```
SERIAL1_PROTOCOL = 2
SERIAL1_BAUD     = 57
SERIAL2_PROTOCOL = 11
SERIAL2_BAUD     = 460
BRD_SER2_RTSCTS  = 0
SERIAL5_PROTOCOL = 2
SERIAL5_BAUD     = 921
```

If `BRD_SER5_RTSCTS` appears, set it to 0 too.

5.3 Calibration on the **Sensors** and **Radio** pages:

1. **Accelerometer:** follow the six orientations, then **Level Horizon** with the drone flat.
2. **Compass:** rotate the drone through every axis until it finishes, then reboot.
3. **Radio:** transmitter on, SBUS receiver bound. Calibrate, moving every stick and switch to its limits.

5.4 Flight modes. Flight Modes page, on a 3-position switch:

Position	Mode	Purpose
1	Stabilize	Full manual
2	Loiter	GPS hold, your normal takeover mode
3	RTL	Come home

Flipping this switch during autonomous flight immediately replaces GUIDED. That is your manual override.

5.5 Battery monitor (6C power port 1; values from the ArduPilot 6C page), on the **Power** page or in Parameters:

```
BATT_MONITOR    = 4
BATT_VOLT_PIN   = 8
BATT_CURR_PIN   = 4
BATT_VOLT_MULT  = 18.182
BATT_AMP_PERVLT = 36.364
```

Reboot, then compare QGC's battery voltage with a multimeter on the battery. Adjust `BATT_VOLT_MULT` if they differ by more than 0.1 V.

5.6 Motor outputs (DShot600 on AUX 1–4). The ESCs stay wired exactly as under PX4, and DShot needs no ESC calibration. On the 6C, MAIN 1–8 are `SERVO1` – `SERVO8` and AUX 1–8 are `SERVO9` – `SERVO16`. Functions 33–36 are Motor 1–4, and PX4 and ArduPilot number quad X motors the same way (1 front-right, 2 rear-left, 3 front-left, 4 rear-right), so these values copy your PX4 mapping:

```
MOT_PWM_TYPE     = 6
SERVO1_FUNCTION  = 0
SERVO2_FUNCTION  = 0
SERVO3_FUNCTION  = 0
SERVO4_FUNCTION  = 0
SERVO9_FUNCTION  = 35
SERVO10_FUNCTION = 34
SERVO11_FUNCTION = 36
SERVO12_FUNCTION = 33
```

`MOT_PWM_TYPE` = 6 is DShot600. Reboot, connect the flight battery (props still off), then open the **Motors** page:

1. Test each motor at low throttle. ArduPilot's letters A, B, C, D go clockwise from the front-right motor.
2. Check each letter spins the motor in that position, in the direction ArduPilot's quad X diagram shows: front-right and rear-left counter-clockwise, front-left and rear-right clockwise.
3. Wrong motor spins: swap the `SERVO9` – `SERVO12` values. Wrong direction: swap any two of that motor's three wires, or reverse it in the ESC configurator.
4. Repeat until all four are right. Disconnect the battery.

5.7 Starting tune. QGC has no equivalent of Mission Planner's Initial Tune Parameters helper. Use [ArduPilot Methodic Configurator], which runs on Linux and works out the starting filter and controller values from your props and battery. Install it in its own environment inside your repo (ignored by Git, so only your parameter files get committed), and close QGC first so it can reach the USB port. Later runs need only the activate line and the last line:

```
cd $REPO
grep -qx '.venv-amc/' .gitignore || echo '.venv-amc/' >> .gitignore
python3 -m venv $REPO/.venv-amc
source $REPO/.venv-amc/bin/activate
pip install --upgrade pip
pip install ardupilot_methodic_configurator
ardupilot_methodic_configurator
```

Enter 10-inch props and a 4S battery when it asks for your components, and apply its initial values. Alternatively, boot Windows once and use Mission Planner's Initial Tune Parameters, then save the parameters to your repo.

5.8 Failsafes and fence. On the **Failsafes** page or in Parameters. Starting values for a 4S pack:

```
FS_THR_ENABLE   = 1
BATT_LOW_VOLT   = 14.0
BATT_CRT_VOLT   = 13.2
BATT_FS_LOW_ACT = 2
FS_GCS_ENABLE   = 1
FENCE_ENABLE    = 1
FENCE_TYPE      = 7
FENCE_RADIUS    = 100
FENCE_ALT_MAX   = 50
```

That is RTL on RC loss, low battery or ground-station loss, inside a 100 m radius, 50 m ceiling geofence. Copter 4.7 renamed some position parameters to metres; if a name doesn't appear, search for its `_M` variant.

Phase 5 passes when: no setup page is marked red, the motor test is right for all four motors, and the only pre-arm warning left is about proximity (fixed in Phase 6). Save `$REPO/params/s500_ardu_05_base.params` and commit.

Phase 6: Lidar and obstacle avoidance

The goal: a lidar picture that matches reality in every direction, then avoidance on with the values already proven in SITL. Props off. The serial port (TELEM2 = SERIAL2) was set in Phase 5.

6.1 Enable the lidar (ArduPilot RPLidar page):

```
PRX1_TYPE   = 5
PRX1_ORIENT = 0
```

Reboot. You should feel a slight vibration as the C1 spins up.

6.2 Check it's alive. QGC has no radar view, so use **Analyze Tools → MAVLink Inspector →** **`DISTANCE_SENSOR`**. ArduPilot reports the lidar as distance readings by direction; the `orientation` field says which: 0 front, 2 right, 4 back, 6 left (odd numbers are the diagonals). If there are no `DISTANCE_SENSOR` messages, or you get "PreArm: check proximity sensor": check TX/RX aren't swapped, `SERIAL2_PROTOCOL` = 11, `SERIAL2_BAUD` = 460, and the C1 has 5 V at VCC. Optional: Mission Planner on Windows shows the same data as a radar picture (Ctrl+F → Proximity).

6.3 Check the directions. Hold a box or board about 1.5 m from the drone and watch which `DISTANCE_SENSOR` orientation drops to about 1.5 m:

- [x] ~~In front → shows in front.~~
- [x] ~~On the right → shows on the right.~~
- [x] ~~Behind → shows behind.~~
- [x] ~~On the left → shows on the left.~~

Everything rotated by a fixed angle means the C1's cable isn't pointing to the rear: set `PRX1_YAW_CORR` or remount. Left and right swapped means `PRX1_ORIENT` is wrong.

6.4 Mask the airframe. With nothing around, any permanent red lines are parts of the S500. For each one:

```
PRX1_IGN_ANG1 = <angle of the obstruction, degrees>
PRX1_IGN_WID1 = <width to ignore, degrees>
```

Use `IGN_ANG2` / `IGN_WID2` and so on for more. Keep them narrow: the drone is blind inside them. Remounting clear of the frame is better than masking.

6.5 Turn on avoidance (same values as SITL):

```
AVOID_ENABLE = 7
AVOID_MARGIN = 2
AVOID_BEHAVE = 1
OA_TYPE      = 1
```

Refresh the parameters (**Parameters → Tools → Refresh**); the `OA_BR_` parameters appear only after `OA_TYPE` is set. Then:

```
OA_BR_TYPE      = 1
OA_BR_LOOKAHEAD = 5
OA_MARGIN_MAX   = 2
OA_DB_ALT_MIN   = 2
GUID_OPTIONS    = 64
```

Reboot. BendyRuler won't run until the Pixhawk restarts.

Parameter	What it does
`AVOID_ENABLE` = 7, `AVOID_MARGIN` = 2, `AVOID_BEHAVE` = 1	Simple avoidance: stop 2 m from obstacles. Works **only in Loiter and AltHold**

Parameter	What it does
`OA_TYPE` = 1	BendyRuler path planning in GUIDED, AUTO and RTL (BendyRuler page)
`GUID_OPTIONS` = 64	Makes GUIDED use path planning, so BendyRuler applies to your scripts' position targets
`OA_BR_LOOKAHEAD` = 5	Look 5 m ahead when planning
`OA_MARGIN_MAX` = 2	Keep 2 m from obstacles while planning
`OA_DB_ALT_MIN` = 2	Ignore obstacles near home below 2 m, so the ground isn't one during takeoff

In GUIDED, BendyRuler is the only avoidance, and only for **position targets**. Velocity commands (WASD-style) get none.

Phase 6 passes when: the radar view matches reality in all four directions, the frame is clear or masked, pre-arm checks pass, and the parameters above are set. Save `$REPO/params/s500_ardu_06_lidar.param` and commit.

Phase 7: Raspberry Pi 5 on TELEM3

The goal: the Pi reads the Pixhawk's heartbeat over TELEM3 and sends a body-relative move you've already proven in SITL. Based on ArduPilot's Raspberry Pi via MAVLink guide, with the Pi 5 differences added.

7.1 Install the OS. Raspberry Pi Imager → **Raspberry Pi OS Lite (64-bit)**. In Imager's settings set the hostname (`s500-pi`), your username, bench Wi-Fi, and enable SSH. Boot and SSH in.

7.2 Enable the GPIO UART. On the Pi 5, GPIO 14/15 is `/dev/ttyAMA0` and must be switched on; `/dev/serial0` points at the small debug connector instead.

```
sudo raspi-config
```

Interface Options → Serial Port → login shell over serial: **No** → serial port hardware: **Yes**. Then:

```
echo "dtparam=uart0=on" | sudo tee -a /boot/firmware/config.txt
sudo usermod -a -G dialout $USER
sudo reboot
```

After the reboot, `ls -l /dev/ttyAMA0` should show the port.

7.3 Software and your repo. The Pi doesn't need ArduPilot's source, and a plain clone skips the submodule. Add the Pi's SSH key to GitHub first (a read-only deploy key on the repo is enough if it only pulls).

```
sudo apt update && sudo apt install -y python3-venv git
python3 -m venv ~/drone-venv
source ~/drone-venv/bin/activate
pip install pymavlink MAVProxy
git clone git@github.com:Antonio9381/TronV2ardrupilot.git ~/tronV2adrupilot
```

7.4 First contact (Pixhawk on USB power, Pi on its own supply, TELEM3 wired as in Phase 4):

```
source ~/drone-venv/bin/activate
mavproxy.py --master=/dev/ttyAMA0 --baudrate 921600
```

MAVProxy should print the flight mode and load parameters. Stuck at "Waiting for heartbeat": TX/RX swapped, ground missing, `SERIAL5_PROTOCOL` / `SERIAL5_BAUD` not 2/921, or `dtparam=uart0=on` missing.

7.5 The relative-move script. Save as `$REPO/pi/nudge.py`. It finds the Pixhawk's own heartbeat, refuses unless the drone is in GUIDED, and caps any move at 5 m. With `GUID_OPTIONS` = 64 these moves are planned around obstacles too.

```
import sys
from pymavlink import mavutil

conn = sys.argv[1]
fwd, right, down = (float(v) for v in sys.argv[2:5])
if max(abs(fwd), abs(right), abs(down)) > 5:
    sys.exit("move larger than 5 m, refusing")

m = mavutil.mavlink_connection(conn, baud=921600, source_system=1,
source_component=191)
while True:
    hb = m.recv_match(type="HEARTBEAT", blocking=True)
```

```
    if hb.autopilot == mavutil.mavlink.MAV_AUTOPILOT_ARDUPILOTMEGA:
        m.target_system = hb.get_srcSystem()
        m.target_component = hb.get_srcComponent()
        break

mode = mavutil.mode_string_v10(hb)
print("mode", mode)
if mode != "GUIDED":
    sys.exit("not in GUIDED, refusing to send")

m.mav.set_position_target_local_ned_send(
    0, m.target_system, m.target_component,
    mavutil.mavlink.MAV_FRAME_BODY_OFFSET_NED,
    0b110111111000,
    fwd, right, down, 0, 0, 0, 0, 0, 0, 0, 0)
print("sent")
```

Test in SITL first, on the laptop. Add `--out=udp:127.0.0.1:14552` to the Phase 1.4 command, take off with `mode guided`, `arm throttle`, `takeoff 10`, then:

```
apenv
python3 $REPO/pi/nudge.py udpin:0.0.0.0:14552 0 2 0
```

The drone moves 2 m to its right. Repeat towards a post: it should route around it rather than hit it.

On the real Pi (props off, disarmed): `python3 ~/tronV2adrupilot/pi/nudge.py /dev/ttyAMA0 0 2 0` should print the mode and refuse. That proves the link and the safety check; the real move happens in Phase 9.

Phase 7 passes when: MAVProxy on the Pi sees the Pixhawk, `nudge.py` works in SITL, and on the real Pi it reads the mode and refuses outside GUIDED.

Phase 8: Ground station (QGroundControl + MAVProxy + your scripts)

Only one program can open the SiK radio's serial port. MAVProxy opens it and shares the link: QGroundControl on UDP 14550, your scripts on 14551. This is the same arrangement SITL already uses, so the scripts don't change.

8.1 Install QGroundControl stable. Download the stable AppImage from the QGroundControl website (not Daily), then:

```
chmod +x QGroundControl-x86_64.AppImage
./QGroundControl-x86_64.AppImage
```

In QGC's **Application Settings → General → AutoConnect**, untick **SiK Radio**. Otherwise QGC grabs the radio's port before MAVProxy can. Leave **UDP** ticked.

8.2 Start the router. Plug in the ground SiK radio first, then:

```
apenv
ls /dev/ttyUSB*
mavproxy.py --master=/dev/ttyUSB0 --baudrate 57600 --out udp:127.0.0.1:14550
--out udp:127.0.0.1:14551
```

Use the device name `ls` showed. When MAVProxy shows the flight mode, open QGroundControl; it connects to UDP 14550 automatically. Keep the MAVProxy terminal open for the whole flight: closing it cuts QGC and the scripts off.

8.3 The field target script. Save as `$REPO/gcs/target.py`. Unlike `goto.py`, it never arms or takes off: it sends one target only when the drone is already flying in GUIDED.

```
import sys, math
from pymavlink import mavutil

conn = sys.argv[1]
lat, lon, alt = (float(v) for v in sys.argv[2:5])

m = mavutil.mavlink_connection(conn)
while True:
    hb = m.recv_match(type="HEARTBEAT", blocking=True)
    if hb.autopilot == mavutil.mavlink.MAV_AUTOPILOT_ARDUPILOTMEGA:
        m.target_system = hb.get_srcSystem()
        m.target_component = hb.get_srcComponent()
        break

mode = mavutil.mode_string_v10(hb)
armed = bool(hb.base_mode & mavutil.mavlink.MAV_MODE_FLAG_SAFETY_ARMED)
if mode != "GUIDED" or not armed:
    sys.exit(f"refusing: mode {mode}, armed {armed}")

m.mav.set_position_target_global_int_send(
    0, m.target_system, m.target_component,
    mavutil.mavlink.MAV_FRAME_GLOBAL_RELATIVE_ALT_INT,
    0b110111111000, int(lat * 1e7), int(lon * 1e7), alt,
```

```
    0, 0, 0, 0, 0, 0, 0, 0)

while True:
    p = m.recv_match(type="GLOBAL_POSITION_INT", blocking=True)
    dn = (lat - p.lat / 1e7) * 111320
    de = (lon - p.lon / 1e7) * 111320 * math.cos(math.radians(lat))
    d = math.hypot(dn, de)
    print(f"{d:6.1f} m to go, alt {p.relative_alt / 1000:5.1f} m")
    if d < 2:
        break
print("arrived")
```

Test it in SITL first: take off manually in MAVProxy (`mode guided`, `arm throttle`, `takeoff 10`), then, in another terminal:

```
apenv
python3 $REPO/gcs/target.py udpin:0.0.0.0:14551 <lat> <lon> 10
```

In the field the command is identical, with the router from 8.2 running.

8.4 Where the agent fits. Once Phase 9 passes, the LLM planner goes in front of these scripts: sentence → objective dictionary → validated in plain code (geofence, altitude, known targets) → `target.py`-style GPS targets from the ground station, and `nudge.py`-style relative moves from the Pi's mission manager. QGroundControl stays on screen the whole time as your view and override.

Phase 8 passes when: MAVProxy holds the radio link, QGC shows the drone through it, and `target.py` works against SITL and refuses on the real drone while disarmed.

Phase 9: Flight tests

Each flight adds exactly one new thing; move on only when the previous one passes. Fly in an open field, within CASA's rules and your club's field rules, with a second person watching.

Every flight: one hand near the mode switch. Anything unexpected → **Loiter** (or Stabilize if GPS is the problem), land, read the log before flying again.

1. **Manual hover.** Props on for the first time. Take off in Stabilize, hover at 2–3 m, then try Loiter. *Passes:* no oscillation, holds position in Loiter, all four motors balanced in the log.

2. **Failsafes.** At 10 m in Loiter, switch to RTL: it comes home and lands. Then, with a spotter, turn the transmitter off briefly: RC failsafe RTL. Switch it back on and retake control with the mode switch. *Passes:* both RTLs behave, and you can always retake control.
3. **Lidar in Loiter.** Fly slowly towards a large soft obstacle (foam board, inflatable, a wall with plenty of room). *Passes:* stops about 2 m away however you push the stick.
4. **Go to location from QGC.** Take off in Loiter to 10 m. In QGC, click the flight mode in the top toolbar and choose **Guided**, then click the map 30–50 m away with nothing in between → **Go to location** and confirm the slider. *Passes:* flies there and holds.
5. **Go to location past one obstacle.** Lower the waypoint speed first (`WPNAV_SPEED` or its `_M` equivalent in 4.7, about 2 m/s). Put the obstacle on the line. *Passes:* bends around it, keeping about 2 m away.
6. **Target from your script.** Same as 5, but with `target.py` through the MAVProxy router (Phase 8). *Passes:* "arrived", with avoidance.
7. **Pi relative moves.** Hover in GUIDED at 10 m, SSH into the Pi from close range, run `nudge.py /dev/ttyAMA0 0 2 0`. *Passes:* moves about 2 m right; repeated towards the obstacle, it routes around it.
8. **Ground station link loss in GUIDED.** Unplug the ground radio during a target flight. *Passes:* RTL after the GCS failsafe timeout.
9. **Only now: the agent.** Start the mission manager and planner with simple objectives (one target, then target plus search) before adding camera tracking.

After each passing flight, download the log (QGC: **Analyze Tools → Log Download**) into `$REPO/logs/` with that day's parameter file. For plots, open the `.bin` log in UAV Log Viewer (plot.ardupilot.org) or MAVExplorer.

Once the basics are solid, run AutoTune at a safe height for a better-tuned S500, especially before adding a camera payload.

Flight-day checklist

The same steps every time you fly, in order. Print it or keep it open on your phone at the field.

1. At home, the day before

- [ ] Charge: flight batteries, transmitter, laptop, and the Pi's supply if it has its own pack.
- [ ] Laptop: `cd $REPO && git pull`, so you have the latest scripts.

- [ ] Pi, while it still has home Wi-Fi: `cd ~/TronV2ardrupilot && git pull`.
- [ ] Pi network for the field: there's no home Wi-Fi there, so let the Pi join your phone's hotspot. Set this up once, at home, with the hotspot switched on: `sudo nmcli dev wifi connect "<hotspot name>" password "<hotspot password>"`. The laptop joins the same hotspot at the field, so you can SSH in.
- [ ] Optional dress rehearsal: run the day's script against SITL (Phase 1.4 command, then the script on `udpin:0.0.0.0:14551`).

2. At the field: drone and transmitter

- [ ] Props on, each on the correct motor (front-right and rear-left counter-clockwise, front-left and rear-right clockwise), and tight.
- [ ] Battery strapped down. Lidar mast clear, GPS arrow pointing forward, nothing loose near the props.
- [ ] Transmitter switches **before** power: CH5 on Stabilize or Loiter, **CH6 (GUIDED) low**, **CH7 (arm) disarmed**, **CH8 (emergency stop) low**.
- [ ] **Transmitter on first**, then plug in the flight battery. Leave the drone still on level ground for about 30 seconds while the sensors initialise.

3. Laptop: terminal 1, the radio router

Plug the ground SiK radio into the laptop, then:

```
apenv
ls /dev/ttyUSB*
mavproxy.py --master=/dev/ttyUSB0 --baudrate 57600 --out udp:127.0.0.1:14550
--out udp:127.0.0.1:14551
```

Use the device `ls` showed. Wait until MAVProxy shows the flight mode and loads parameters. Leave this terminal open for the whole flight: closing it cuts off QGC and the scripts.

4. Laptop: QGroundControl

Start the stable AppImage. It connects through UDP 14550 by itself. Check in this order:

- [ ] Top bar says **Ready**, not "Not Ready". If not, read the pre-arm messages; don't override them.
- [ ] GPS: **10+ satellites**, HDOP under about 1.0, and the home **H** on the map where the drone is.

- [ ] Battery voltage about right for a charged 4S pack (16.4–16.8 V).
- [ ] Heading: the drone's arrow on the map points the way its nose points.
- [ ] Lidar: **Analyze Tools → MAVLink Inspector →** `DISTANCE_SENSOR` readings look sensible, with nothing closer than you expect.
- [ ] Flight mode shown matches your CH5 switch, and flipping CH6 changes it to GUIDED and back. Leave CH6 low afterwards.

5. Laptop: terminal 2, the Pi (if this flight uses it)

```
ssh wrylands@joeyberry.local
source ~/drone-venv/bin/activate
cd ~/TronV2ardrupilot
```

Don't run MAVProxy on the Pi while a Pi script needs `/dev/ttyAMA0`: only one program can hold the port.

6. Laptop: terminal 3, the ground station script (if this flight uses it)

```
apenv
cd $REPO/gcs
```

Have the command typed and ready, e.g. `python3 target.py udpin:0.0.0.0:14551 <lat> <lon> 15`. Don't press Enter until the drone is hovering in GUIDED.

7. Flying a mission

1. Spotter in place, area clear, everyone behind you.
2. Press the **safety button** until solid, then **arm** with CH7.
3. Take off in **Stabilize** or **Loiter**, and climb to about 10 m. Check it holds position steadily in Loiter.
4. Flip **CH6 high**: the mode becomes **GUIDED**.
5. Start the script (terminal 3, or terminal 2 for the Pi). Keep one hand on the switches throughout.
6. When it reports arrived or finishes, flip **CH6 low**: you're back in Loiter. Land.
7. **Disarm** with CH7, then unplug the flight battery.

If anything looks wrong:

Situation	Do
Script misbehaving, drone wandering	**CH6 low**: instantly back to manual Loiter
You're unsure where it is or it's drifting away	CH5 to **RTL**
About to hit someone or something	**CH8 emergency stop**: motors cut, the drone falls. Last resort only
Script needs stopping	Ctrl+C in its terminal; the drone holds the last target. Then CH6 low

8. After the flight

- [ ] Battery unplugged before you stop anything on the laptop.
- [ ] Stop the scripts and MAVProxy with Ctrl+C.
- [ ] Download the log: QGC **Analyze Tools → Log Download**, newest log, saved into `$REPO/logs/`.
- [ ] If you changed any parameters, save them to `$REPO/params/` with the date and what changed.
- [ ] Commit and push: `cd $REPO && git add params gcs pi && git commit -m "Flight YYYY-MM-DD: what you tested" && git push`.
- [ ] Write one line in your notes on what worked and what to change. Then tick the flight off in Phase 9.

Troubleshooting

Everything that has come up so far, and the fix.

Symptom	Cause	Fix
`git push` rejected, "fetch first"	GitHub repo was created with its own commit (README)	`git pull origin main --allow-unrelated-histories`, resolve any conflict, then `git push -u origin main`
Install script stalls on wxPython	extras.wxpython.org not responding	Ctrl+C; wxPython 4.2.1 is already installed system-wide

Symptom	Cause	Fix
`No such file or directory: 'mavproxy.py'`	ArduPilot venv not active	`apenv` before `sim_vehicle.py`
`apenv: command not found`	Alias not loaded in that terminal	`. ~/.bashrc`
`[ble: detached]`, garbled prompt after a build	Prompt add-on confused by long output	Close the terminal and open a new one
Prompt shows `detached at APMrover2-stable`	Another tag on the same commit	Check `git -C $REPO/ardupilot tag --points-at HEAD` lists Copter-4.7.2
`can't convert ... to float`, `Usage: takeoff ...`	Several MAVProxy lines pasted at once	Type one line at a time
`Unable to find parameter 'OA_BR_LOOKAHEAD'`	Set before `param fetch` finished	Wait for `Received ... parameters`, then set it
`PreArm: BendyRuler OA requires reboot`	`OA_TYPE` changed	`reboot` (SITL) or reboot the Pixhawk
Drone flies straight through obstacles in GUIDED	`GUID_OPTIONS` not 64	`GUID_OPTIONS` = 64, reboot
Settings gone after restarting SITL	Started with `-w`	Never use `-w` after the first run
QGC Firmware page has no ArduPilot option	QGC Daily build	Use QGC stable, or the uploader (Phase 3)
`Permission denied` on `/dev/ttyACM0` or `/dev/ttyUSB0`	dialout group not active yet	Log out and back in
Uploader can't see the board	QGC or ModemManager holds the port	Close QGC; `sudo apt remove -y modemmanager`

Symptom	Cause	Fix
"PreArm: check proximity sensor"	No lidar data	TX/RX, `SERIAL2_PROTOCOL` = 11, `SERIAL2_BAUD` = 460, 5 V at the C1
Pi stuck at "Waiting for heartbeat"	Link not set up	TX/RX, ground, `SERIAL5_PROTOCOL` / `SERIAL5_BAUD` = 2/921, `dtparam=uart0=on`
MAVProxy can't open the radio in the field	QGC auto-connected to it first	Untick SiK Radio in QGC's AutoConnect; start MAVProxy before QGC
QGC shows no vehicle in the field	Router not running, or UDP autoconnect off	Start the 8.2 router; tick UDP in QGC's AutoConnect

Sources

ArduPilot documentation:

- Holybro Pixhawk 6C: UART mapping, battery values, firmware folder
- Guided mode: `GUID_OPTIONS` bit 6 enables path planning in GUIDED
- Object avoidance with BendyRuler
- Simple object avoidance: Loiter and AltHold only
- RPLidar A1/A2/C1/S1 setup
- Communicating with Raspberry Pi via MAVLink
- Setting up the build environment and SITL on Linux
- Simulated peripherals: simulated 360° lidar and posts
- ArduPilot Methodic Configurator: starting tune values on Linux
- Copter stable firmware
- Optional Mission Planner extras: loading firmware, advanced tools (proximity view)

Other sources:

- Copter-4.7.2 release announcement (ArduPilot forum)
- Pixhawk6C board definition: board ID 56
- Pixhawk 6C port pinouts (Holybro)
- Pixhawk 6C power limits (PX4 docs)
- RPLIDAR C1 specifications (Waveshare)